

# QPU Bench

Open Source Quantum Computing Benchmark Framework

---

MQS · mark@mq.s.dk

2026

Molecular Quantum Solutions

# Framework Overview

---

# What is QPUBench?

## Modality-agnostic quantum benchmark framework

- Separates *what* you benchmark (schemas) from *how* execution happens (adapters) and *how results are stored* (stores)
- Typed **Pydantic v2** schema layer — zero quantum SDK dependencies in core
- Gate-based circuits, MBQC on FPGA, molecular VQE, evolutionary circuit search, and error-mitigated runs all share one BenchmarkRecord
- Schema **v1.12.0** — 21 modules, language-agnostic JSON output

### Core invariant

qpubench itself never imports from any quantum library. Only adapters do.

## Core layer (*no SDK dependencies*)

- schemas/ — 21 Pydantic v2 modules
- runner.py — BenchmarkRunner
- store.py — NDJSONStore, ParquetStore

## Integration layer (*your project*)

- backends/ — adapter protocols + stubs
- integrations/ — copy into your project
- examples/ — runnable demos

CircuitSpec / Problem → Adapter → BenchmarkRecord → Store

# Key Design Principles

| Principle                      | Implementation                                                  |
|--------------------------------|-----------------------------------------------------------------|
| <b>Schema-first</b>            | Pydantic v2 is the stable contract — all records are valid JSON |
| <b>No SDK lock-in</b>          | Core never imports any quantum library                          |
| <b>Three adapter protocols</b> | BackendAdapter · AlgorithmAdapter · ErrorMitigationAdapter      |
| <b>Append-only stores</b>      | NDJSON and Parquet for reproducible sweeps                      |
| <b>Language-agnostic</b>       | Any language can parse the NDJSON output                        |

# Installation

*# Minimal – schemas + stubs, no quantum SDK required*

```
pip install .
```

*# With optional backends*

```
pip install ".[qiskit]"      # Qiskit Aer + IBM Quantum Runtime V2
```

```
pip install ".[storage]"    # Parquet store (pyarrow + pandas)
```

```
pip install ".[all]"        # everything on PyPI
```

**Package managers:** pip · uv · Poetry 2 · conda

**Credentials:** via .env file and BackendSpec.auth (never hardcoded)

## Schema Layer

---

## Core Schema Modules

| Module                        | Modality         | Key types                                                                                                     |
|-------------------------------|------------------|---------------------------------------------------------------------------------------------------------------|
| <code>primitives</code>       | <code>all</code> | <code>QPUModality</code> , <code>CircuitFormat</code> , <code>PauliLabel</code> , <code>ComplexNumber</code>  |
| <code>circuit</code>          | <code>all</code> | <code>CircuitSpec</code> — <code>from_openqasm3()</code> , <code>bind()</code> , <code>is_parametric()</code> |
| <code>observable</code>       | <code>all</code> | <code>SparsePauliObservable</code> , <code>PauliTerm</code>                                                   |
| <code>backend</code>          | <code>all</code> | <code>BackendSpec</code> — 27 factory constructors                                                            |
| <code>execution</code>        | <code>all</code> | <code>ExecutionOptions</code> , <code>AlgorithmSpec</code> , <code>ZNEConfig</code>                           |
| <code>error_mitigation</code> | <code>all</code> | <code>ErrorMitigationStrategy</code> , <code>provider configs</code> .                                        |



## Domain-Specific Schema Modules

| Module           | Modality / Framework                                                                      |
|------------------|-------------------------------------------------------------------------------------------|
| mbqc             | MBQC-FPGA — bit-exact 16-bit program word                                                 |
| photonic         | Linear-optics chips, FBQC, HOM, photonic VQE/analog                                       |
| gbs              | Gaussian Boson Sampling — hafnian, vibronic, TDM/Borealis                                 |
| qdk_chemistry    | SCF → active space → QPE → resource estimation                                            |
| qse              | Krylov Quantum Diagonalization (KQD / SQD)                                                |
| qesem            | QESEM (Qedma) — noise scaling, QET, device characterisation                               |
| qcschema         | QCSchema / QCElemental / PennyLane interoperability                                       |
| neutral_atom     | Bloqade / Aquila AHS, Rydberg atom arrangement                                            |
| slowquant        | SlowQuant UCC/VQE — ansatz, SCF, linear response                                          |
| error_mitigation | Q-CTRL · Mitiq · Haiqu · ParityQC · QMatter · Quantum Motion · IBM V2 · Advantage Tracker |

# QPU Modalities

QPUBench natively covers **9 quantum computing paradigms**:

- GATE\_BASED — qubit circuits (QASM2/3)
- MBQC — measurement-based QC on FPGA
- PHOTONIC\_LINEAR\_OPTICS — Fock-state chips
- FUSION\_BASED — FBQC resource states + fusion
- QPE — Quantum Phase Estimation / IQPE
- GBS — Gaussian Boson Sampling
- KQD — Krylov Quantum Diagonalization
- NEUTRAL\_ATOM — AHS on Rydberg atoms
- ANNEALING — quantum annealing

All modalities share the same BenchmarkRecord schema.

## CircuitSpec — One Type for All Circuits

*# Gate-based (QASM2)*

```
circuit = CircuitSpec(  
    num_qubits=2,  
    serialized='OPENQASM 2.0;\ninclude "qelib1.inc";\n...',  
    observables=[SparsePauliObservable(num_qubits=2, terms=[  
        PauliTerm(qubit_indices=(0, 1),  
                    pauli_ops=(PauliLabel.Z, PauliLabel.Z),  
                    coefficient=ComplexNumber(re=1.0))  
    ])],  
)
```

*# OpenQASM 3.0 helper*

```
circuit = CircuitSpec.from_openqasm3(source, num_qubits=2)
```

# Adapters

---

# Three Adapter Protocols

**BackendAdapter** *Circuit in,  
result out*

CircuitSpec

↓ run()

QuantumResult

Aer · Qrack · IBM IQM ·

MBQC-FPGA

**AlgorithmAdapter** *Problem  
spec in*

CircuitSpec

(problem)

↓ run\_algorithm()

(Result, VQAConfig)

QForte · OpenFermion

**ErrorMitigationAdapter**  
*Wraps a BackendAdapter*

inner adapter

↓ run()

mitigate/suppress

↓

QuantumResult

Fire Opal · Mitiq Haiqu ·

ParityQC

BenchmarkRunner dispatches automatically — ErrorMitigationAdapter registers like any BackendAdapter.

## BackendAdapter — Estimator vs Sampler

```
class MyBackendAdapter:
    @property
    def spec(self) -> BackendSpec: ...           # hardware description

    def validate(self, circuit: CircuitSpec) -> list[str]: ...

    def run(self, circuit: CircuitSpec,
            options: ExecutionOptions) -> QuantumResult:
        if circuit.observables:
            # Estimator path → expectation_values (VQE, QAOA)
            evs = [measure_expectation(circuit, obs) for obs in circuit.observables]
            return QuantumResult(expectation_values=evs, ...)
        else:
            # Sampler path → shot counts / bitstrings
```

## AlgorithmAdapter (QForte example)

```
class QForteAlgorithmAdapter:
    def validate_problem(self, circuit: CircuitSpec) -> list[str]:
        if circuit.format != CircuitFormat.MOLECULE_JSON:
            return [f"Expected MOLECULE_JSON, got {circuit.format!r}"]
        return []

    def run_algorithm(self, circuit: CircuitSpec,
                     options: ExecutionOptions
                     ) -> tuple[QuantumResult, VQAConfig]:
        import qforte as qf                                # ← deferred SDK import
        mol = qf.system_factory(...)
        alg = qf.ADAPTVQE(mol, ...)
        alg.run(pool_type="SD", optimizer="BFGS", ...)
        energy = float(alg.get_gs_energy())
```

## Energy Evaluator Hook

Advanced: QForte drives the ansatz; a qpubench backend evaluates  $\langle H \rangle$

```
QForte optimizer --> energy_feval(params)
                        |
                    EnergyEvaluatorHook.evaluate()
                        |
                    qpubench BackendAdapter.run()
                        |
                    QuantumResult --> <H> (float)
```

- Enables ADAPT-VQE on **Aer, Qrack GPU, IBM hardware**, or any adapter
- Gradient computation automatically routed through the hook
- Falls back to QForte's internal C++ simulator on error
- Available in `integrations/qforte/energy_hook.py`



## Error Mitigation Providers

`error_mitigation.py` — one schema module, seven provider integrations:

| Provider                | Schema types                                                             | Technique                                                   |
|-------------------------|--------------------------------------------------------------------------|-------------------------------------------------------------|
| <b>Q-CTRL Fire Opal</b> | <code>FireOpalConfig</code> ,<br><code>FireOpalResult</code>             | Closed-box noise suppression; <code>fo.execute()</code> API |
| <b>Mitiq</b>            | <code>MitiqConfig</code> + 5 configs,<br><code>MitiqResult</code>        | ZNE · PEC · CDR · REM · DDD                                 |
| <b>Haiqu Rivet</b>      | <code>HaiquRivetConfig</code> ,<br><code>HaiquTranspilationResult</code> | Hardware-aware transpilation caching                        |
| <b>ParityQC</b>         | <code>ParityQCConfig</code> ,<br><code>ParityQCResult</code>             | Parity encoding for QUBO / HCBO / Ising                     |
| <b>QMatter</b>          | <code>QMatterConfig</code> ,<br><code>QMatterCompressionResult</code>    | Quantum problem compression                                 |

# IQM Hardware — Star Topology

**IQM Resonance cloud** (iqm-client + qiskit-iqm)

*# Star architecture: all qubits via COMPR1 resonator*

```
backend = BackendSpec.iqm_resonance(
    "garnet",      # 20Q IQM Garnet
    api_token_ref="IQM_TOKEN",
)
```

*# BackendSpec.iqm\_resonance("deneb") # 6Q*

*# BackendSpec.iqm\_resonance("sirius") # 24Q Star*

```
adapter = IQMAdapter("garnet", token=os.environ["IQM_TOKEN"])
```

Calibration via IQM-

Client.get\_dynamic\_quantum\_architecture():

T1 ~28  $\mu$ s · T2\* ~17  $\mu$ s · PRX ~99.79% ·

CZ ~99.10%

## Native gate set

| Gate | Parameters        | Notes                                                                       |
|------|-------------------|-----------------------------------------------------------------------------|
| PRX  | $\theta, \varphi$ | Angles in <b>fractions of turns</b> — $\theta=0.5 \rightarrow \pi$ rotation |
| CZ   | —                 | Controlled-Z, symmetric                                                     |
| MOVE | —                 | Qubit $\leftrightarrow$                                                     |

# IBM Quantum Runtime V2

```
EstimatorV2 PUB (circuit,
observables, params?, precision?)
# precision replaces fixed shot count
estimator.run([(qc, obs, params, 1e-2)])
result[i].data.evs    # expectation value
result[i].data.stds   # statistical error

SamplerV2 PUB (circuit, params?,
shots?)
sampler.run([(qc, params, 4096)])
bit_array = result[0].data.meas
counts = bit_array.get_counts()
BitArray.shape = (num_shots,) or
(num_params, num_shots)
```

## Execution modes (IBMEExecutionMode)

| Mode    | When to use                             |
|---------|-----------------------------------------|
| SESSION | VQE / adaptive — exclusive QPU hardware |
| BATCH   | Sweeps — parallel independent jobs      |
| SINGLE  | One-shot run (default)                  |

```
adapter = IBMAdapter(
    "ibm_torino",
    execution_mode=IBMEExecutionMode.SESSION
)
```

**IBMRuntimeRecord** on QuantumResult:

job\_id · session\_id · resilience\_level ·

ExecutionSpan timestamps

# Quantum Advantage Tracker

BenchmarkRecord.advantage holds a QuantumAdvantageRecord:

```
record = runner.run(circuit, "ibm_torino", options)
record = record.model_copy(update={"advantage": QuantumAdvantageRecord(
    experiment_type=AdvantageExperimentType.OBSERVABLE_ESTIMATION,
    circuit_name="Loschmidt-echo-70q",
    num_qubits=70,
    backend_name="ibm_boston",
    floquet_layers=20,
    observable_value=0.327,
    observable_error_bound=0.012,
    classical_method=ClassicalComparisonMethod.TENSOR_NETWORK,
    coupling_params={"b": 1.0, "delta": 0.5},
    submission_url="https://github.com/quantum-advantage-tracker/...",
```

```
)))
```

## Execution & Storage

---

## BenchmarkRunner

```
runner = BenchmarkRunner(store=NDJSONStore(Path("results.ndjson")))
runner.register(AerAdapter(), name="aer")
runner.register(QForteAlgorithmAdapter(), name="qforte")
```

*# Single run*

```
record = runner.run(circuit, "aer", ExecutionOptions(shots=4096))
```

*# Cartesian product sweep: circuits × backends × options*

```
records = runner.sweep(
    circuits=[circuit_h2, circuit_lih],
    backend_names=["aer", "qforte"],
    options_list=[ExecutionOptions(shots=1024),
                  ExecutionOptions(shots=8192)],
    run_id="vqe-sweep-001",
```

# Data Persistence

**NDJSONStore** — zero dependencies, append-only, grep-able

```
store = NDJSONStore(Path("results/run1.ndjson"))
store.save(record)

record = store.load(experiment_id)           # by UUID
records = store.query(                       # dot-path filters
    backend__name="aer_statevector",
    result__status="succeeded",
)
```

**ParquetStore** — columnar analytics (pip install "[storage]")

```
store = ParquetStore(Path("results/run1.parquet"))
df = store.to_dataframe()                   # → pandas DataFrame
```

# Integrations

---



## Integration Ecosystem (1/3)

| Integration                    | Schema module    | Protocol         |
|--------------------------------|------------------|------------------|
| Cebule SDK                     | cebule           | BackendAdapter   |
| Xenakis GA circuit search      | xenakis          | AlgorithmAdapter |
| ExcitationSolve (Fourier VQE)  | excitation_solve | AlgorithmAdapter |
| GSOpt (ground-state benchmark) | gsopt            | AlgorithmAdapter |
| Photochipsim / FBQC            | photonic         | BackendAdapter   |
| QDK / QuNorth chemistry        | qdk_chemistry    | AlgorithmAdapter |

## Integration Ecosystem (2/3)

| Integration           | Schema module | Protocol             |
|-----------------------|---------------|----------------------|
| DTU-GBS / photonic_QC | gbs           | BackendAdapter       |
| QSE / KQD             | qse           | AlgorithmAdapter     |
| QESEM (Qedma)         | qesem         | BackendAdapter       |
| QCSchema / PennyLane  | qcschema      | cross-format interop |
| Bloqade / Aquila      | neutral_atom  | BackendAdapter       |
| SlowQuant UCC/VQE     | slowquant     | AlgorithmAdapter     |

## Integration Ecosystem (3/3) — Error Mitigation & Hardware

| Integration                             | Schema module    | Protocol               |
|-----------------------------------------|------------------|------------------------|
| Q-CTRL Fire Opal                        | error_mitigation | ErrorMitigationAdapter |
| Mitiq (ZNE · PEC · CDR · REM · DDD)     | error_mitigation | ErrorMitigationAdapter |
| Haiqu Rivet transpiler                  | error_mitigation | ErrorMitigationAdapter |
| ParityQC parity encoding                | error_mitigation | ErrorMitigationAdapter |
| QMatter problem compression             | error_mitigation | ErrorMitigationAdapter |
| IQM Resonance (garnet · deneb · sirius) | —                | BackendAdapter         |
| Quantum Motion CMOS spin-qubit          | error_mitigation | BackendAdapter         |

## Cross-Framework Compatibility

**Pauli encoding** — explicit converters prevent silent convention bugs:

```
# Qrack Q# convention: I=0, X=1, Z=2, Y=3 (non-sequential!)
```

```
PauliLabel.Z.to_qrack_int()           # → 2 (not 3)
```

```
PauliLabel.Y.to_qrack_int()           # → 3 (not 2)
```

```
# Qiskit C API QkBitTerm bit-packed encoding
```

```
PauliLabel.X.to_qiskit_c_bit_term()   # → 0b0010
```

```
# MBQC byproduct register: bit 0 = Z, bit 1 = X (reversed vs gate-based!)
```

**Complex numbers:** stored as {"re": 1.0, "im": 0.5} — valid JSON without Python eval

**QCSchema / QCElemental / PennyLane** interoperability via the qcschema module

## Code Quality

---

# Python Quality Toolchain

```
ruff check src/ tests/      # lint (line-length 100, target py311)
ruff format src/ tests/     # format
mypy src/                   # static types (strict = true)
pytest tests/ -q            # 156 tests – no quantum SDK required
```

---

| Tool               | Role                                                           |
|--------------------|----------------------------------------------------------------|
| <b>Pydantic v2</b> | Runtime-validated schemas — field types enforced at parse time |
| <b>ruff</b>        | Linting + auto-formatting, replaces flake8/isort/black         |
| <b>mypy strict</b> | Full type coverage — Any only where unavoidable                |
| <b>pytest</b>      | Schema-only test suite runs with pip install . alone           |

## Summary

---

## Getting Started

```
from qpubench import (  
    BenchmarkRunner, NDJSONStore, StubGateAdapter,  
    CircuitSpec, ExecutionOptions,  
    SparsePauliObservable, PauliTerm, PauliLabel, ComplexNumber,  
)  
import pathlib  
  
circuit = CircuitSpec(  
    num_qubits=2,  
    serialized='OPENQASM 2.0;\ninclude "qelib1.inc";\nqreg q[2];\nh q[0];\ncx q[0],  
    observables=[SparsePauliObservable(num_qubits=2, terms=[  
        PauliTerm(qubit_indices=(0, 1),  
                    pauli_ops=(PauliLabel.Z, PauliLabel.Z),  
                    coefficient=ComplexNumber(re=1.0))
```



## Roadmap & Links

**Repository:** `github.com/mqsdk/qpubench`

**Schema v1.12.0** — 21 modules · 9 QPU modalities · zero quantum SDK deps in core

**Contributing** — writing an adapter takes ~30 lines:

```
cp integrations/template/backend_adapter_template.py  my_adapter.py
# fill the three TODOs: spec, validate(), run()
```

See `INTEGRATION_GUIDE.md` and `AGENTS.md` for the full development workflow.

**Contact:** `contact@mq.s.dk`